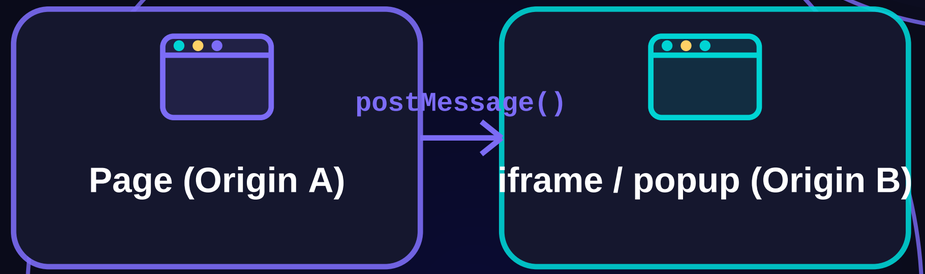
What is postMessage



What is postMessage?

> postMessage is a method that enables cross-origin communication between Window objects.
>
> For example, between a page and a pop-up that it spawned, or between a page and an iframe embedded within it.



Normally, scripts on different pages can only access each other if they share the same origin:

- **Protocol**
- **Host**
- **Port**

Safely bridges different origins

Why Do We Need postMessage?

The same-origin policy keeps scripts on one page from freely reaching into another page on a different origin. postMessage gives you a safe, explicit way around that wall.

Key Benefits

Cross-Origin



Lets a page talk to an iframe or popup on a completely different origin.

Controlled



The receiver checks the sender's origin before trusting any message.

Two-Way



Both windows can send and receive, enabling real two-way communication.

Key Concepts

The pieces you will use every time you work with postMessage



message

The data you want to send to the other window.

```
win.postMessage(data, origin)
```



targetOrigin

The origin the receiving window must have, for safety.

```
"https://example.com"
```



message event

Fired on the receiving window when a message arrives.

```
addEventListener("message", fn)
```



event.origin

The sender's origin; always check it before trusting data.

```
if (e.origin !== trusted)
```

Example: Sending a Message

Suppose a page embeds an iframe. The parent page sends a message to that iframe, naming its exact origin.

parent.js

```
const frame = document.getElementById("myFrame");

frame.contentWindow.postMessage(
  { type: "greet", text: "Hello iframe!" },
  "https://child.example.com"
);
```

popup.js

```
// Sending to a popup window works the same way
const popup = window.open("https://child.example.com");

popup.postMessage(
  { type: "greet", text: "Hello popup!" },
  "https://child.example.com"
);
```

targetOrigin must match the receiver's origin exactly, or the browser won't deliver the message.

Example: Receiving a Message

The receiving window listens for the message event, and should always verify where the message came from.

child.js

```
window.addEventListener("message", (event) => {
  if (event.origin !== "https://parent.example.com") {
    return; // ignore untrusted senders
  }

  console.log(event.data);
  // { type: "greet", text: "Hello iframe!" }
});
```

reply.js

```
// Replying back to the sender
window.addEventListener("message", (event) => {
  if (event.origin !== "https://parent.example.com") return;

  event.source.postMessage(
    { type: "ack", text: "Got it!" },
    event.origin
  );
});
```

event.origin tells you who really sent the message, never trust event.data alone.

Same-Origin vs postMessage

Scenario	Same-Origin Pages	Cross-Origin Pages
Direct JS access	Allowed (window.frame.var)	**Blocked by the browser**
Sharing data	Read each other's variables	**Must use postMessage**
Security check	Implicit, same origin	**You check event.origin yourself**
Typical use case	Same-site iframes	**3rd-party widgets, payment iframes, OAuth popups**

Same origin? Direct access works. Different origin? Reach for postMessage.

Key Takeaways

1. **Cross-origin communication**
   postMessage lets Window objects talk safely across different origins.

2. **Same-origin policy**
   Scripts can only access each other directly when protocol, host and port all match.

3. **Always check the origin**
   Verify event.origin on the receiving end before trusting any message.

4. **Common use cases**
   iframes, popups, OAuth flows, and embedded third-party widgets.